Lecture 01

Course Introduction, Intro to C++

EECS 280

Prof. John Kloosterman

Kloosterfacts

- Prof. Kloosterman
- Ph.D. in CSE from Michigan in computer architecture
- Taught EECS 485 most recently, also 183 and 280
- Course for transfer students in Fall 2024

Who are we?

- Everyone here is a transfer student

- Transfer student can mean lots of things:
  - Community college
  - International
  - 4-year
  - Military

Meeting each other

- 3 rounds of find someone, say hi, what was your path into EECS 280 at Michigan?

What we'll learn in EECS 280

- **Programming skills** in C++
  - Object-oriented programming and how it is implemented

- How data is stored in **memory**
  - Pointers
  - Dynamic memory

- **Data structures**: how to efficiently put data in memory
  - Arrays that can change size
  - Search items without looking at all of them

Role of transfer student section

- Pace of material

- Connection: each other, with me

Kind of learning to expect

- First level: hearing it for first time, memorizing the basics
  - You will do this on your own time before class

- Second level: applying it
  - We'll do this in class with relatively hard problems in partners

Knowing != doing

- Getting back into a kayak
- I have a book about it, but no matter how much I study the book, it won't help me
- I have to actually do the activity

- Grades in EECS 280: based on doing

My goal

- You'll have done harder examples of exam thinking in class
- While there are people here to help
- Then on the projects and exam, you can feel confident you're not doing anything new

My role

- I believe everyone in this room can do well in EECS 280
- I believe it will be a big adjustment for many of you.
- I promise there will be no shame in asking me anything.
- Easiest way to fail: fall behind and feel embarrassed asking to help catching up.

How to reach me

- In this lecture, including during break and afterwards
- My office hours: MW 10-11AM in my office (2630 Beyster)
- E-mail

Course Hints

002 Section Format

- Day before lecture: watch asynchronous videos, do exercises

- Beginning of class: paper diagnostic quiz about async videos
- Review quiz questions
- Review most confusing parts of async videos
- Coding on laptops
- Submit your work to autograder by end of lecture day

Lecture participation

- In this lecture section only, get participation credit
- Formula in syllabus
- I want to give 100% to everybody; this will help your overall grade

- To get the point for the day:
  - do the async exercises by 11:59PM the **day before lecture**
  - be present during lecture
  - hand in lecture code by 11:59PM **the day of lecture**

Participation drops

- Lecture: 3 drops
- Labs: 2 drops (separate)

- You don't need to tell us you're using a drop

Async Textbook Setup

- jkloosterman.net/280_transfer

- Sign up for Github account
- Enter your account in the form
- Verify you can access the textbook

Do all the tutorials!

- Do all the tutorials linked on eecs280.org. Don't treat these as optional
- Computer setup: must be done before next class, with VS Code
- Thursday 6-8PM in LEIN 2420: computer setup help

Grade breakdown

- Median grade is B+
- Almost everyone gets 100% on projects and freebies
  - You should aim to get 95-100% on every project too. Lower means you're in trouble.
- Difference between good/bad grade is exams

EECS 280 Course Components

Logistics for this week

http://eecs280.org

- No lab this week
- Setup help this week
- Office hours start next week
- Project 1 due in ~2 weeks
- Please read the syllabus

Lab

Labs start second week of class

- Attendance at your registered section is required
  - Drop 2 lowest
- Includes
  - Split into small groups of 4 to complete a worksheet
  - Course staff will check in with each group
- Graded for attendance and completion
  - Check in with IA/GSI

Exams

- In-person, on-paper
- Dates on eecs280.org schedule
  - Add to your calendar now
- Extended time
  - Need recommendation the SSD office
  - Submit a request form linked from eecs280.org
- Alternate time
  - Submit a request form linked from eecs280.org

Projects

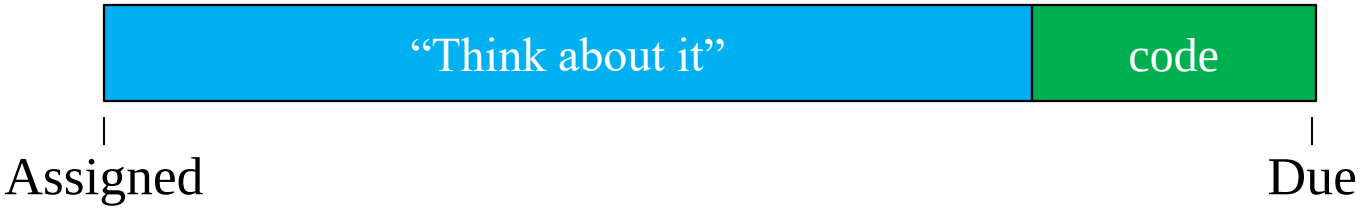
- Build an image processing program, a Euchre card game, a machine learning algorithm, and a text editor
- Get started on Project 1

Project autograder

- 3 submissions per day
  - We grade the best submission
- Public tests
  - Published with the starter files
- Private tests
  - Not published, run after deadline
- Mutation testing
  - Tests of your tests
- Style tests
  - Public

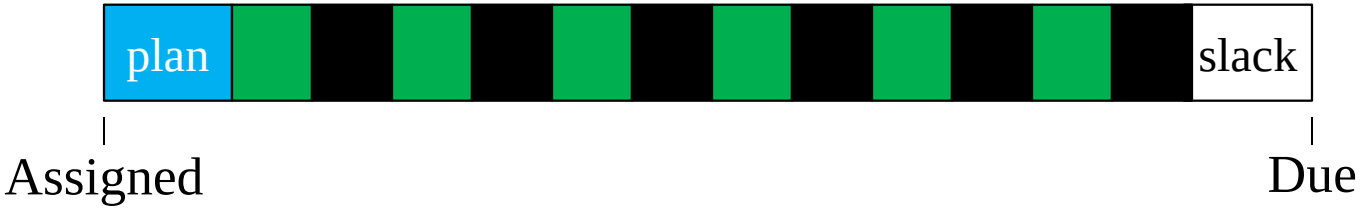
Project timeline

- Sample timeline for failure on difficult projects

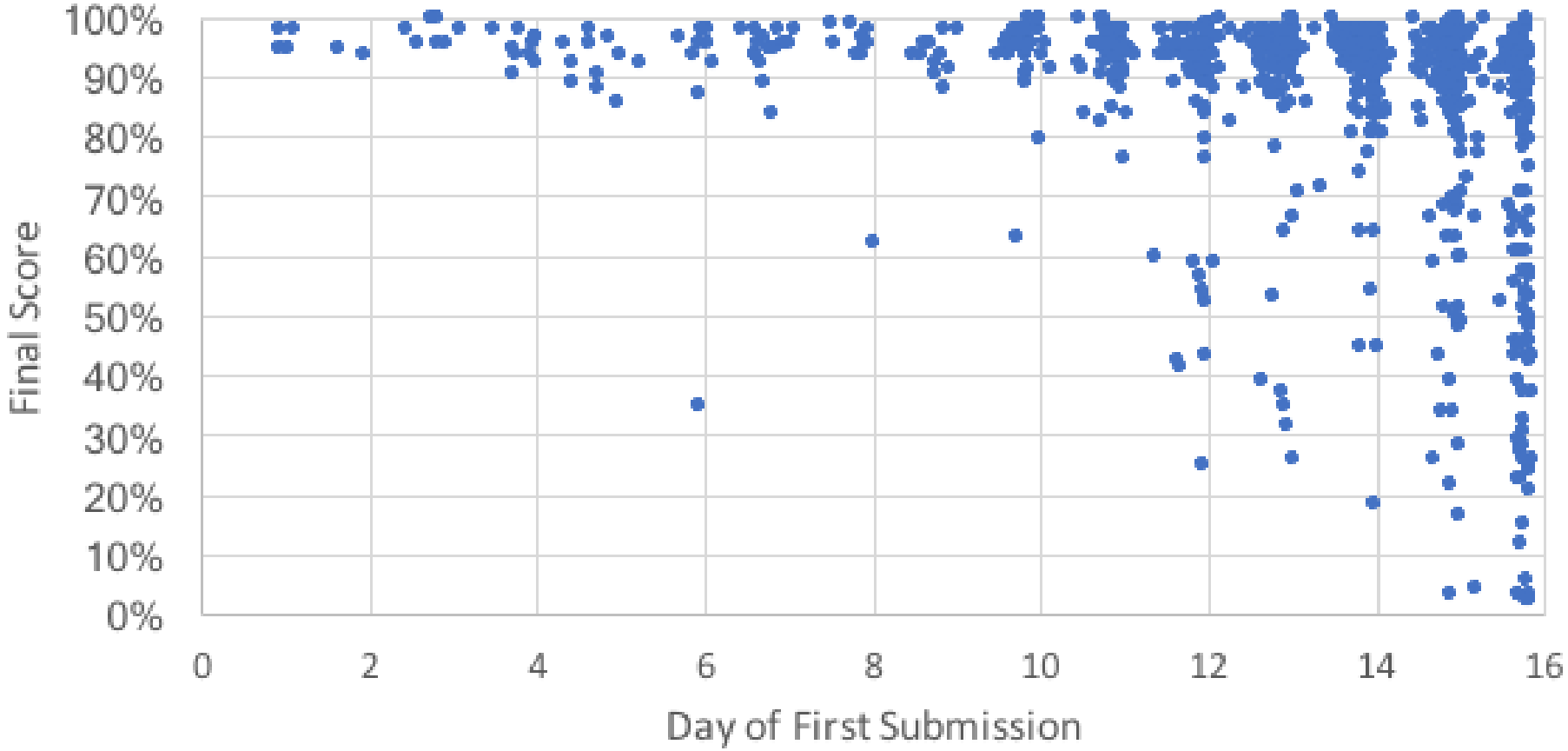


- Sample timeline for success on difficult projects

- Even better



Initial submission vs. final score



Agenda

- Orientation
  - Course components
  - **Grades**
  - Getting help
- Machine model
  - Code, programs, and variables
  - Value semantics and reference semantics
  - Call stacks

Grades

- Letter grades assigned on a straight scale
- Exams weighted less

Assignment	Weight
Lab (all labs equal weight)	10%
Projects (P1=4%, P2=8%, P3=9%, P4=5%, P5=9%, P6=4%)	39%
Midterm Exam	25%
Final Exam	25%
Entry and Exit surveys	1%

Please see the syllabus for full details

Exam curve and minimum scores

- After each exam, we may adjust your score to account for the difficulty of the exam
  - Goal is a mean in the mid 80's
  - Never adjust scores downward
- To pass EECS 280 with a C or better
  - Total weighted project score must be at least 60.0%, and
  - Total weighted curved exam score must be at least 65.0%

Collaboration

Task	Collaboration
Project 1	No
Labs	Your lab group
Projects 2 – 5	Partners only
Exams	No

Please see the syllabus for more details

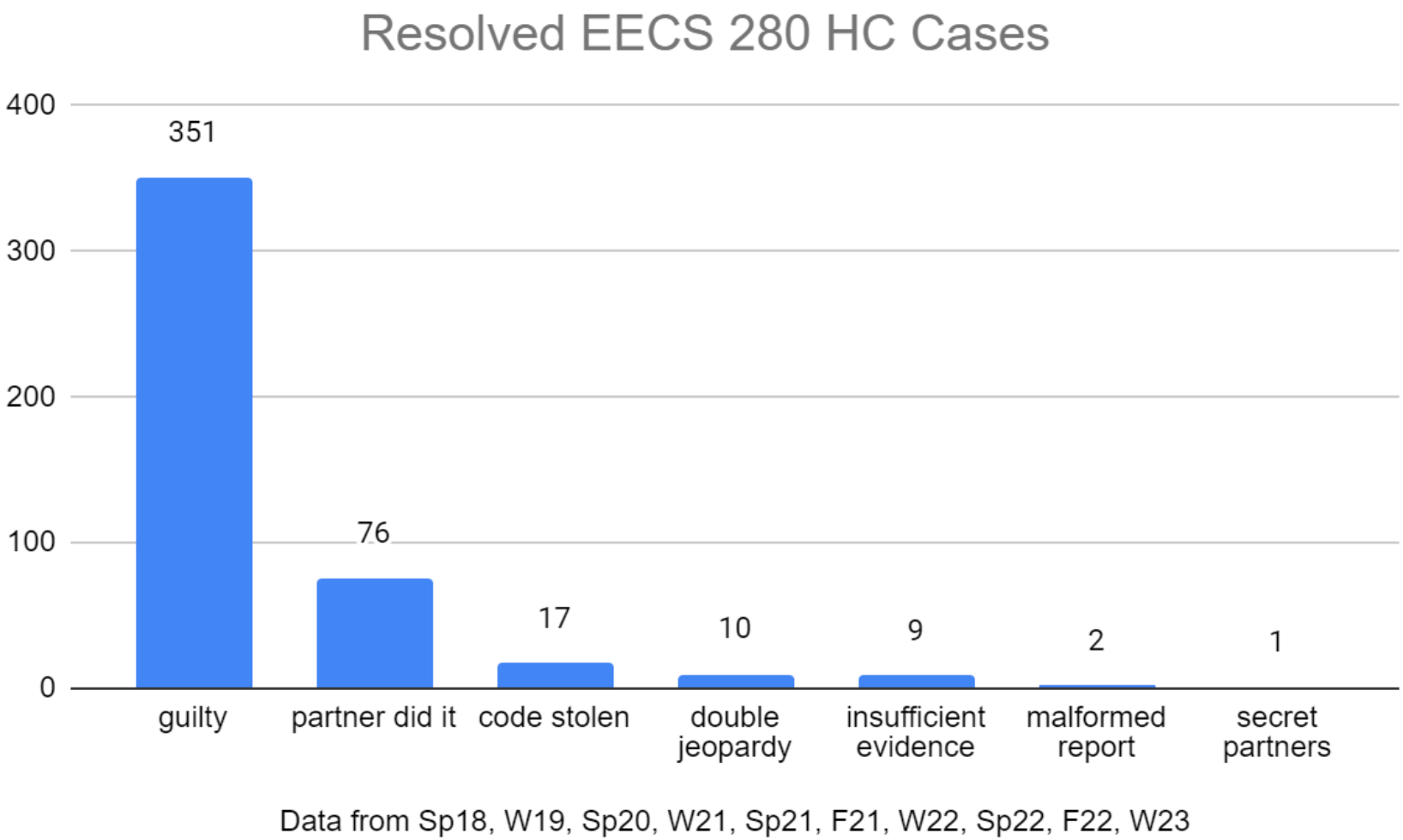
Project collaboration

Encouraged	Prohibited
Share **high-level** design strategies	Share **step-by-step** code, **pseudocode**, or **comments**
Help others **understand** the project	**Share** your solution as a "reference"
Help others debug	Debug someone's code **for them**
Explain a compiler error to someone	Fix a compiler error **for someone**
Discuss testing **strategies**	**Share** test case code (even if not submitted)
Brainstorm edge case tests together	Tell others what **specific tests** they need to pass the autograder
Using starter code or example code given in lecture	**Copy** code in whole or in part, or write code for someone else
Looking at someone else's code to **understand** concepts	**Share** your code in a way that could be **copied**

Cheat checking

- Automated software and manual inspection
- Over the past several years, we have reported 2-10% of the class each semester.
- We only report **clear cases** of academic dishonesty.
- You will **not** be reported for:
  - Using starter code provided by course instructors.
  - Having the same idea as someone else.
  - Receiving similar help/guidance from the same course staff member in office hours.

Cheat-Checking



Generative AI Policy

- The use of generative AI tools is permitted only in the same contexts where collaboration with another person is allowed.

Allowed	Prohibited
Reinforcing course concepts, generating additional examples, interactively checking knowledge.	Generating project code, test cases, or any other work to be submitted (with or without modifications)
Explain compiler errors or get suggestions for debugging	Exhaustively proofreading your code to find and/or correct any mistakes
Brainstorming and identifying potential edge cases for testing	Asking for a list of test cases (even if they aren't written out as code)

Generative AI rationalizations

- ***"This is just a minor/silly assignment"***

- ***"I'll use AI to generate a solution only as a way to learn the material."***

Prof. Kloosterman on GenAI

- People use GenAI and cheat not because they are cheaters but because they feel overwhelmed
  - Project is due tomorrow but didn't plan time well
- You have the resources here to address the underlying problem
  - Talk to me!
  - Early while we can do something about it!
- I recommend not touching GenAI at all for 280: it's temptation
  - I'll forgive you, but you're forming bad habits
  - It will be harder to break bad habit than to face difficulty now

Agenda

- Orientation
  - Course components
  - Grades
  - **Getting help**
- Machine model
  - Code, programs, and variables
  - Value semantics and reference semantics
  - Call stacks

Staff office hours

- Project planning and conceptual questions
- Debugging support
- Sign up linked from eecs280.org

Proffice hours

- “Professor Office Hours”
- Need to talk to an instructor about the course?
- Check eecs280.org for times

Ed

- Online forum
- There's a welcome message that explains the guidelines: don't post code publicly, be kind

Agenda

- Introduction
- Course policies
- **C++ fundamentals**

Warmup question

- Say hi to partner

Warmup question

- What does this code print?
- Make a guess about where one **expression** is.

```
int main() {
  int i1 = 200;
  int i2 = 40;

  cout << "Welcome to EECS "
       << i1 + i2 * 2 << endl;
}
```

Tracing

```
#include <iostream>

int main() {
  // print a greeting
  std::cout << "Hello World!" << std::endl;

  // Declare some variables
  int x = 10 + 5;
  int y = 0;
  y = x;
  x = 20;

  // Print out the result
  std::cout << "x=" << x << " y=" << y << std::endl;
}
```

Compiler

Machine code

- Each CPU uses a specific Instruction Set Architecture (ISA) that defines its machine language.

hello.exe

X86_64

```
push    rbp
mov     rbp, rsp
mov     DWORD PTR [rbp-4], 3
mov     DWORD PTR [rbp-8], 4
mov     edx, DWORD PTR [rbp-4]
mov     eax, DWORD PTR [rbp-8]
add     eax, edx
...
```

hello.exe

AArch64

```
sub     sp, sp, #16
mov     w0, 3
str     w0, [sp, 12]
mov     w0, 4
str     w0, [sp, 8]
ldr     w1, [sp, 12]
ldr     w0, [sp, 8]
add     w0, w1, w0
...
```

Compiling vs. Interpreting

Compile errors

- If there are any compile errors, no hello.exe file is created.

hello.cpp

```
#include <iostream>

int main() {
  // print a greeting
  std::cout << "Hello World!" << std::endl;

  // Declare some variables
  int x = 10 + 5;
  int y = 0;
  y = x
  x = 20;

  // Print o
  std::cout
}
```

This is a syntax error.

```
jjuett@LAPTOP-N6606F3M:~/temp$ g++ hello.cpp -o hello.exe
hello.cpp: In function ‘int main()’:
hello.cpp:6:33: error: expected ‘;’ before ‘int’
    6 |   cout << "Hello World!" << endl
      |                                  ^
      |                                  ;
......
    9 |   int x = 10 + 5;
      |   ~~~
hello.cpp:11:7: error: ‘x’ was not declared in this scope
   11 |   y = x;
      |       ^
```

Statements

```
#include <iostream>
using namespace std;

int main() {
    int x = 3
    int y = 4;
}
```

```
test3.cpp: In function ‘int main()’:
test3.cpp:6:4: error: expected ‘,’ or ‘;’ before ‘int’
    6 |     int y = 4;
      |     ^~~
```

- Use the word "statement" to describe what went wrong here
- How many statements are inside main()?

Printing output

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello 280";
    cout << "Hello again" << endl;
}
```

- What will print? Do you think it is what the programmer expected?
- Describe what you think "cout" (pronounced see-out) and "endl" mean.

Namespaces

```
#include <iostream>

int main() {
  // print a greeting
  std::cout << "Hi!" << std::endl;
}
```

```
#include <iostream>
using namespace std;

int main() {
  // print a greeting
  cout << "Hi!" << endl;
}
```

- Both of these programs mean the same thing
- cout is in the **namespace** std
- Describe why the "using namespace std" means we don't need to put "std::" in front of cout.

Formatting

- Spaces and line breaks don't affect functionality.
- They DO make your code more readable.

```
#include <iostream>
using namespace std;

int main() {
  // print a greeting
  cout << "Hello World!" << endl;

  // Declare some variables
  int x = 10 + 5; int y = 0; y = x; x = 20;




  // Print out the result
        cout << "x=" << x << " y=" << y << endl;
}
```

Several statements on one line

Inconsistent indentation

Comments

- // The rest of the line is a comment
- /* and */ Any code between is a comment
- IDEs have keyboard shortcuts for comment/uncomment

```
#include <iostream>
using namespace std;

int main() {
  // print a greeting
  cout << "Hello World!" << endl; // another comment

  // Declare some variables
  int x = 10 + 5;
  int y = x;
  /*y = x;
  x = 20;*/

  // Print out the result
  cout << "x=" << x << " y=" << y << endl;
}
```

Expressions

```
int main() {
  int x = 3;
  int y = ;
}
```

```
test3.cpp: In function ‘int main()’:
test3.cpp:6:12: error: expected primary-expression before ‘;’ token
    6 |     int y = ;
      |             ^
```

- Explain in clearer language why this program makes no sense
- What kinds of things can go before the semicolon?

Variables and types

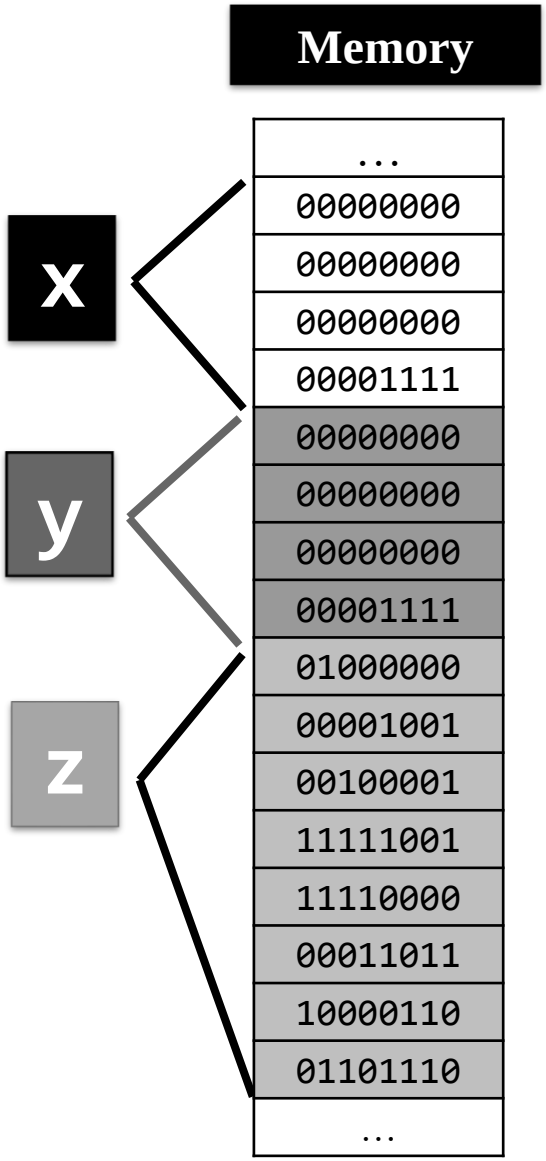
```
int main() {
    string s = "Hello";
    int x;
    x = s;
}
```

- Why doesn't the last statement work?
- What does this tell you about the **types** of variables and how they can/can't change?

Variables and memory

- A variable is a name in the program's source code that refers to an object in memory.

```
// Declare some variables
int x = 10 + 5;
int y = x;
double z = 3.14159;
```



- Data is stored in **bytes**, composed of 8 **bits** - binary digits of 0 or 1.

- A variable's **type** determines:
  - How much space it needs
    e.g. `int` = 4 bytes, double = 8 bytes
  - How its value is encoded to binary

Uninitialized variables

- Warning! Initialize variables before using them.
  - Unfortunately, the compiler can't always check this for you.

```
cout << x; // COMPILE ERROR

// x is uninitialized - has some "random" value
int x;

// Runtime Error - Compiles successfully, but
// has undefined behavior when the program runs.
cout << x;

x = 3; // OK – x now has value 3
cout << x; // Behavior is now well-defined
```

- Some errors can be caught at compile time whereas others don't necessarily occur until runtime.

Scope

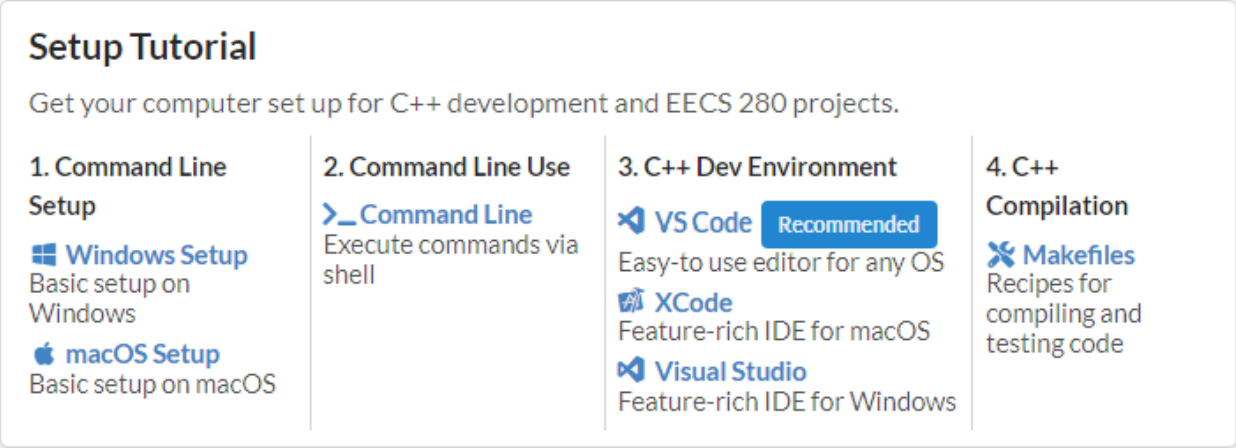
```
int square(int x, int y) {
   y = x * x;
   x = 1;
   return y;
}

int main() {
  int x = 5;
  int y;
  int z = square(x, y);

  cout << x;
  cout << y;
  cout << z;
}
```

- What will this program print?
- Use the word **scope** to describe why x's value is 5 at the end of the program and not 1.

Your todo list

- Verify exam times and dates. Schedule.
- Add project deadlines and exams to your calendar. Schedule.
- Sign up for a CAEN account if you don't have one
- Read the syllabus
- Do the setup tutorial
- Start Project 1

Setup help session this week!



002: Due for next lecture

- Computer setup finished
  - Required to do the in-lecture exercises next class
  - Use VS Code
  - Verify that AI completion turned off

- Lectures 1 and 2 on async textbook due Sunday, Jan. 11 at 11:59PM